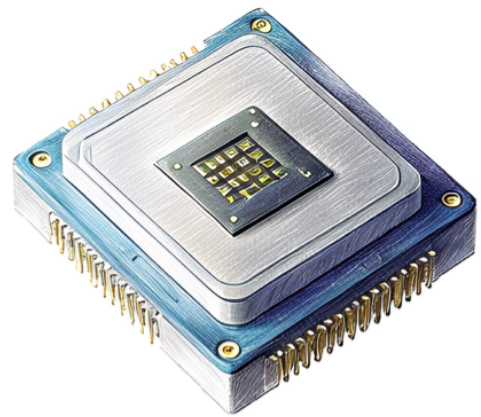


PROCESSOR ARCHITECTURE

Modern microprocessors are amongst the most complex systems ever created.



The instructions supported by a particular processor and their byte-level encodings are known as its ISA.

The ISA provides a conceptual layer of abstraction between compiler writers and chip designers.

NOTE - The way a processor operates can be different from the model of computation implied by the ISA. For example, the ISA can imply sequential execution of all operations in a program. However, a modern processor can execute multiple instructions parallelly to improve performance. Special mechanisms are used to ensure consistency in end results.

↳ This chapter uses a Y86-64 ISA defined below.

* Defining an ISA -

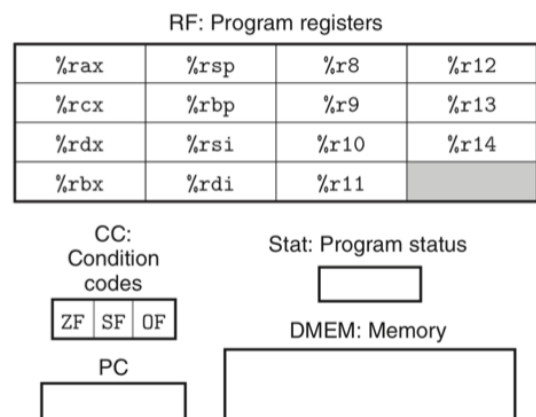
↳ This involves defining the various components of its state, the set of instructions and their encodings, a set of programming conventions and the handling of exceptional events.

Each instruction in a Y86-64 program can read and modify some part of the processor state → This is called programmer visible state.

↳ assembly code writer or the compiler

The state of Y86-64 is similar to x86-64 -

1. There are 15 program registers.
2. Each register stores a 64 bit word.
3. %rsp is used as the stack pointer.
4. There are three single bit condition codes.
5. The PC holds the currently executed instruction.
6. Memory is a large array of bytes holding program & data
↳ Y86-64 programs use virtual addresses.
7. The status code STAT indicates the overall status of execution.



The Y86-64 instruction set -

1. The move instructions - `movq`
 - ↳ `s` → represents the source. (Can be immediate (`i`), register (`r`) or memory (`m`))
 - ↳ `d` → represents the destination. (Can be register (`r`) or memory (`m`))
 - ↳ memory to memory transfers are disallowed like x86.
2. Integer Operations - `addq`, `subq`, `andq`, `xorq`
 - ↳ Only operate on register data
 - ↳ x86 allows integer operations directly on memory.

3. There are seven jump instructions, and six conditional move instructions.
4. Operations associated with the stack -
 - ↳ call ↳ pushq
 - ↳ ret ↳ popq
5. The halt instruction → stops the processor and sets status code to HLT
 - ↳ x86 programs have a similar operation but aren't permitted to use it.
 - ↳ causes the entire system to suspend.

Instruction Encoding - * Instruction encoding needs to be unique.

- Each instruction requires between 1 and 10 bytes.
- The first byte identifies the instruction type



The function values are relevant only when a group of related instructions share a common code.

- The 15 program registers have associated register IDs (0x00 to 0x0F)
 - ↳ The numbering matches x86-64.
 - ↳ Stored within the CPU in a register file.
 - ↳ Small random access file where register ID's serve as addresses.
 - ↳ The address 0x0F is used to indicate that no register is to be used.
- Instructions which require operands have a register specifier byte
 - ↳ Type | ID1 | ID2
 - ↳ Instructions with 1 operand have register ID2 set to 0x0F.
 - ↳ Instructions with no operands have no register specifier byte.
- Some instructions have an additional 8 byte word for immediate values.
 - ↳ Y86 uses absolute addresses as opposed to PC relative like x86.
 - ↳ Makes processor design a lot simpler.
 - ↳ All integers have little endian encoding, just as x86.

Exceptions -

- The programmer visible state includes a status code describing the overall status of the executing program.
- We use simple error handling → stop instruction execution on non-OK status.
- In x86, there are dedicated exception handlers which can trap into the OS.

* RISC vs CISC -

Complete set of differences can be found in the CS:APP textbook.

CISC -

- Large number of instructions.
- Some instructions have long execution times → Memory copies
- Variable sized instruction encodings.
- Multiple formats for operand specification.

RISC -

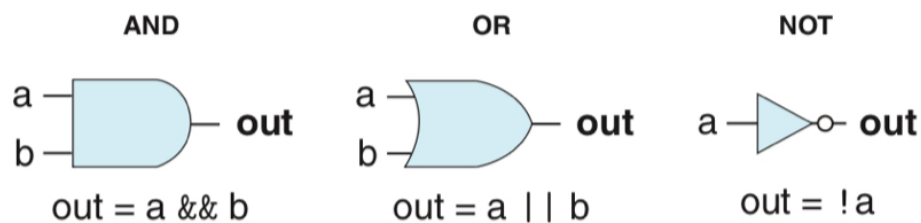
- Fewer instructions - typically < 100
- No instruction with long execution times
- Fixed length encoding → 4 byte encodings.
- Simple addressing format.
 - ↳ often just base & displacement.

* Logic Design -

Components required to implement a digital system -

1. Combinational logic to compute functions
2. Memory elements to store bits.
3. Clock signals to regulate memory element updates.

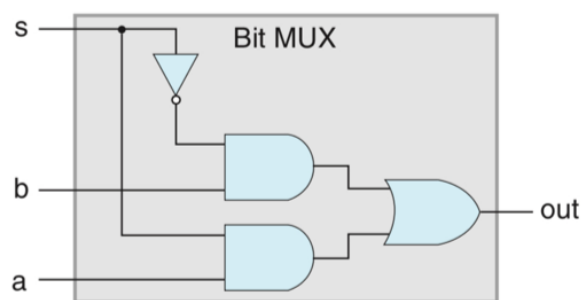
Logic gates are the basic building blocks of digital circuits. Logic gates operate on single bit quantities, and not words. Logic gates are always active. If some input to a gate changes, then within some small amount of time, the output will change as well.



By assembling a number of logic gates into a network, we can construct combinational circuits. Rules to create a combinational circuit are -

1. No floating wires - Every input must connect to a valid source.
2. No fighting outputs - Outputs of two gates cannot be wired together.
3. No infinite loops - Networks must be acyclic

Figure 4.11
Single-bit multiplexor circuit. The output will equal input a if the control signal s is 1 and will equal input b when s is 0.



A multiplexor is an example of a combinational circuit. It selects a value from among a set of different data signals, depending on the value of a control input signal.

Both C and combinational circuits use boolean functions, with the following differences -

1. Outputs of combinational circuits continuously respond to changes in inputs. C expressions are only evaluated when they are encountered in the program.
2. Logical C expressions can be partially evaluated. Combinational logic do not support partial evaluations.

Very large networks of logic gates can be used to create combinational circuits that perform word-level computations. One such circuit is the ALU. The ALU is an advanced mux which performs different operations based on a control signal.

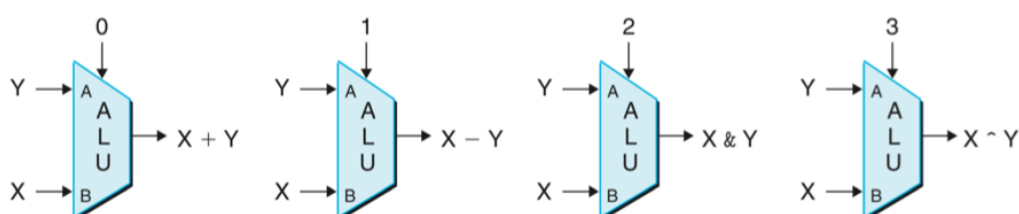
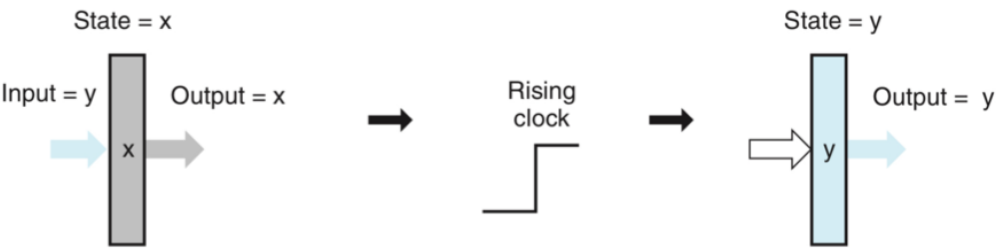


Figure 4.15 Arithmetic/logic unit (ALU). Depending on the setting of the function input, the circuit will perform one of four different arithmetic and logical operations.

Combinational circuits by their very nature, do not store any information. They simply react to inputs. To create systems that have state, we must have devices that can store information. The storage devices are controlled by a single clock, which determines when new values are to be loaded in. There are two classes of memory devices-

- 1. Clocked Registers
- 2. Random Access Memory

Registers mean slightly different things in hardware & programming. In hardware a register is directly connected to the rest of the circuit by its input and output wires. In programming, registers represent a small collection of addressable words in the CPU. The addresses consist of register IDs. The register IDs are stored in the register files.



→ Values only propagate from a register input to its output once every clock cycle at its rising edge.

Figure 4.16 Register operation. The register outputs remain held at the current register state until the clock signal rises. When the clock rises, the values at the register inputs are captured to become the new register state.